RESEARCH-ARTICLE

How Does Software Prefetching Work on GPU Query Processing?

YANGSHEN DENG, Southern University of Science and Technology, Shenzhen, Guangdong, China

SHIWEN CHEN, Southern University of Science and Technology, Shenzhen, Guangdong, China

ZHAOYANG HONG, Southern University of Science and Technology, Shenzhen, Guangdong, China

BO TANG, Southern University of Science and Technology, Shenzhen, Guangdong, China

Published: 09 June 2024

Citation in BibTeX format

SIGMOD/PODS '24: International Conference on Management of Data
June 10, 2024
AA, Santiago, Chile

Conference Sponsors:
SIGMOD

How Does Software Prefetching Work on GPU Query Processing?

Yangshen Deng
Department of Computer Science and Engineering,
Southern University of Science and Technology
AlayaDB AI
Shenzhen, Guangdong, China
dengys2022@mail.sustech.edu.cn

Shiwen Chen
Department of Computer Science and Engineering,
Southern University of Science and Technology
AlayaDB AI
Shenzhen, Guangdong, China
chensw2023@mail.sustech.edu.cn

Zhaoyang Hong
Department of Computer Science and Engineering,
Southern University of Science and Technology
AlayaDB AI
Shenzhen, Guangdong, China
hongzy2022@mail.sustech.edu.cn

Bo Tang*
Department of Computer Science and Engineering,
Southern University of Science and Technology
AlayaDB AI
Shenzhen, Guangdong, China
tangb3@sustech.edu.cn

ABSTRACT

Improving the performance of GPU query processing is a well-studied problem in database community. However, its performance is still unsatisfactory due to the low utilization of GPU memory bandwidth. In the literature, employing software prefetching techniques to improve the bandwidth utilization is a common practice in CPU database as it overlaps computation cost and memory access latency. However, it was ignored by GPU database even though the software prefetching ability has been provided by modern GPU architecture (i.e., from NVIDIA Ampere).

In order to investigate the effectiveness of software prefetching techniques on GPU query processing, we implement four software prefetching algorithms on GPU, i.e., Group Prefetch (GP), Software-Pipelined Prefetch (SPP), Asynchronous Memory Access Chaining (AMAC) and Interleaved Multi-Vectorizing (IMV) in the work. We then adapt them on hash join probe and BTree search tasks with a suite of optimizations. Last, we conduct comprehensive experiments and evaluate the performance of them. The results confirm the superiority of software prefetching techniques on GPU query processing. Specifically, they can achieve up to 1.19X speedup on hash join probe and 1.31X speedup on BTree search when compared with the implementations without software prefetching.

ACM Reference Format:
Yangshen Deng, Shiwen Chen, Zhaoyang Hong, and Bo Tang. 2024. How Does Software Prefetching Work on GPU Query Processing?. In *20th International Workshop on Data Management on New Hardware (DaMoN '24), June 10, 2024, Santiago, AA, Chile.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3662010.3663445

*Dr. Bo Tang is the corresponding author.

DaMoN '24, June 10, 2024, Santiago, AA, Chile

ACM ISBN 979-8-4007-0667-7/24/06
https://doi.org/10.1145/3662010.3663445

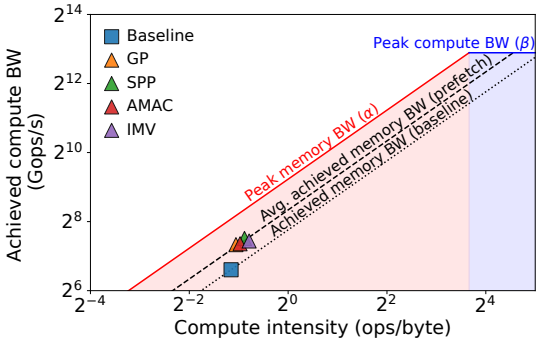


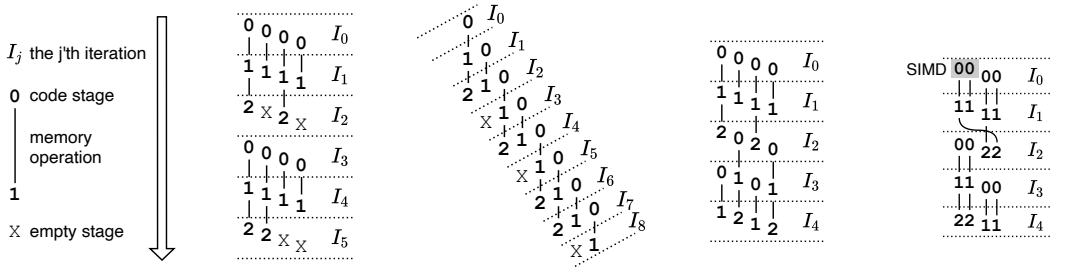
Figure 1: Analyzing BTree search performance via roofline

1 INTRODUCTION

Many works utilize Graph Processing Units (GPUs) to accelerate query processing in database community [8, 12, 13, 16, 18, 28, 42, 46, 47]. In the literature, the GPU is usually used as a co-processor of the host CPU to accelerate the computation intensive tasks during the query processing, e.g., join algorithms [33, 40, 45], sorting algorithms [29, 36, 41], and aggregation operations [22, 46]. Recently, with the fast development of GPU capability, it has been used as the primary processor in many GPU database systems (a.k.a., GPUDB) [17, 34, 39].

In this work, we focus on the latter case: the GPU is the primary processor. Even though many techniques have been proposed to improve the performance of the query processing in GPUDB, e.g., tile-based query execution [39], lane-refill and push-down parallelism [17], and Shuffle and Segment operator [34], the GPUDB query processing performance still has enough room to improve as the achieved memory bandwidth [9, 39] of them is obviously smaller than the peak memory bandwidth of GPU. The achieved memory bandwidth shows the memory level parallelism (MLP) degree. Existing work of GPU query processing improves the MLP by workload-unconscious *hardware scheduling*. Specifically, it overlaps the computation operations and memory accesses implicitly.

To confirm the improvement space of the query processing in GPUDBs, We utilize roofline [44] to analyze the searching performance of BTree. Specifically, we implement a BTree on NVIDIA A10



(a) GP ($G = 4$) (b) SPP ($D = 1$) (c) AMAC ($G = 4$) (d) IMV ($G = 2, W = 2$)

Figure 2: Illustration of prefetching algorithms on CPUDB

GPU. It contains 256M key-value pairs and each key and each value are 4-byte. We measure the performance of concurrently searching all keys on it with one CUDA kernel. Roofline model is a well-known performance-modeling method to analyze the performance bottleneck and resource utilization of the running algorithms [9]. Conventionally, every running algorithm on the hardware will correspond to a point in the colored area in Figure 1, i.e., it is either memory-bound or compute-bound, see the red line α and the blue line β. The performance of algorithms cannot be improved when they saturate the bandwidth of either memory or compute. The blue square in Figure 1 shows the searching performance of BTree, which is the baseline method and uses hardware scheduling implicitly. Obviously, it confirms that: (i) BTree search is memory-bound, and (ii) memory bandwidth is significantly underutilized, i.e., its MLP is very low.

In the database community, dozens of works have been proposed to improve the memory bandwidth utilization (i.e., MLP) by explicit *software prefetching* techniques in CPU [10, 15, 24]. Interestingly, the modern GPU architectures (e.g., NVIDIA Ampere) provide prefetching ability by allowing asynchronous memory copy from global memory to shared memory. Inspired by the success of software prefetching on CPUDB, we explore the effectiveness of software prefetching on GPUDB. To the best of our knowledge, this problem has not been studied in our community. We start by implementing four prefetching algorithms on GPU, i.e., Group Prefetch (GP) [10], Software-Pipelined Prefetch (SPP) [10], Asynchronous Memory Access Chaining (AMAC) [24] and Interleaved Multi-Vectorizing (IMV) [15]. All of them are widely used in traditional CPUDB. However, it is not trivial to implement them on GPU. The are two major reasons: (i) the prefetching mechanism on GPU is different from that on CPU, and (ii) GPU does not provide SIMD instructions to implement vectorization for IMV. We address these challenges and adapt these four algorithms to two typical tasks: hash join probe and BTree search. In addition, we devise several GPU-specific optimizations to reduce memory access and alleviate divergence inside a code stage during the execution. We next conduct extensive experiments to analyze their performance. The results confirm that the software prefetching can gain up to 1.19X and 1.31X speedup in hash join probe and BTree search on GPUDB. As the triangles shown in Figure 1, the achieved average memory bandwidth of the prefetching-enabled BTree search is obviously better than the baseline. Last but not least, we propose a guideline to efficiently use prefetching on GPUDB via the observed insights from experimental results.

Notation	Description
P	The number of code stages in a code path.
G	The group size in GP, AMAC and IMV.
D	The prefetch distance in SPP.
W	The width of vectors in IMV.

Table 1: Notations for prefetching algorithms

2 PRELIMINARIES

Query processing in databases is executing a specific code path for each input tuple. For example, probing a hash table is searching the bucket chains and comparing the key of the input tuple with the keys in buckets. The code path can be divided into a sequence of dependent *code stages*. Each stage performs several compute operations and a memory access. We refer the number of code stages in a code path as the length of the code path, denoted as P. In this section, we use an example in Figure 2 to illustrate the four prefetching algorithms on CPUDB.

2.1 Group Prefetching

The procedure of Group Prefetch (GP) [10] is shown in Figure 2(a). It gathers each G input tuples into a group and processes tuples group by group. In this example $G = 4$. Each iteration processes a stage of all tuples in the group and issues prefetching instructions for the memory access in this stage. For example, iteration I_0 processes stage 0 of the 4 tuples. In this way, memory accesses and compute operations are overlapped inside a group, e.g., memory access of the first tuple is overlapped with the compute operations in stage 0 of the other 3 tuples. We can always find a large enough G to enable fully overlapping. In order to buffer the states of all tuples in a group, GP needs a memory space of $O(G)$.

The limitation of GP is the lack of ability to handle irregular code paths. For example, the 2nd and 4th tuples do not have stage 2, which results in a shorter code path than the 1st and 3rd tuples. There are not enough compute operations to hide the latency of the second memory access from the 3rd tuple. Therefore, the variable-length code paths will result in low MLP.

2.2 Software-Pipelined Prefetching

Different from GP that processes one stage in an iteration, the Software-Pipelined Prefetch (SPP) [10] forms a pipeline that processes all code stages in an iteration. Given a prefetch distance

(a) Packing DVS into RVS (b) Filling DVS with RVS

Figure 3: Reorganization of vector states ($W = 4$)

Characteristic	GP	SPP	AMAC	IMV
Executed stages per iteration	single	all	dynamic	dynamic
The size of state buffer	$O(G)$	$O(PD)$	$O(G)$	$O(GW)$
Handle divergent code paths			✓	✓
SIMD				✓

Table 2: Prefetching algorithms summary

D [31], the next stage of a tuple is processed D iterations away. Figure 2(b) shows an example with $D = 1$. Stage 1 of the 1st tuple is processed in iteration I_1 and its next stage is processed in I_2. In this way, compute operations and memory accesses are overlapped across iterations, e.g., the first memory access of the 3rd tuple is overlapped with stages 0, 1, and 2 of the 4th, 2nd, and 1st tuple. To buffer the states of all tuples that are being processed, SPP requires a memory space of $O(PD)$.

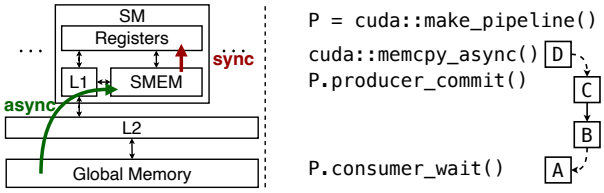
SPP has the same limitation in handling irregularities as GP. For example, since stage 2 of the 2nd tuple is empty, there are not enough compute operations to overlap with the first memory access of the 4th tuple.

2.3 Asynchronous Memory Access Chaining

Asynchronous Memory Access Chaining (AMAC) [24] was proposed to address the limitations of GP and SPP in handling irregularities. Like GP, AMAC processes G tuples in an iteration and requires $O(G)$ memory to buffer the tuple states. Figure 2(c) shows an example with $G = 4$. AMAC maintains the stage information of each tuple and dynamically selects a stage to process. When it encounters an empty stage, it fills the empty stage with the first stage of the next input tuple to maintain the MLP. For example in iteration I_2, AMAC fills the empty stage 2 of the 2nd tuple with the stage 0 of the 5th tuple. In this way, there are still enough compute operations to overlap with the 2nd memory access of the 3rd tuple.

2.4 Interleaved Multi-Vectoring

IMV was proposed to leverage the Single Instruction Multiple Data (SIMD) instructions in modern processors. SIMD instructions process a vector with W element at a time, where W is the width of the vector. IMV processes G vectors in an iteration. A memory space of $O(GW)$ is required to buffer the tuple states. Figure 2(d) shows an example with $W = 2$ and $G = 2$. Each 2 tuples are grouped into a vector and processed by SIMD instructions. When the code stages of tuples within a vector differ, only a part of the vector with the same stages can be processed at a time, which underutilizes the parallelism of SIMD. The stages of elements in this vector form a divergence vector state (DVS). For example, after the iteration I_1, stages of the 1st and 2nd tuples are 2 and X, which form a DVS "2X". Stages of the 3rd and the 4th tuple also form the same DVS.



(a) GPU Memory Architecture (b) CUDA Pipeline APIs

Figure 4: Asynchronous memory copy on GPU

To overcome this limitation, IMV proposes a method to reorganize the DVSs into fully vectorized states, where all tuples in a vector have the same stages. In our example, IMV reorganizes the two previously generated DVSs "2X" into a fully vectorized states "22", and processes it with SIMD in iteration I_2.

We use an example with $W = 4$ to show the implementation of this reorganization, see Figure 3. The bold font represents the states after each reorganization. First, a buffer is allocated to gather the residual states in DVSs. This buffer is called residual vector state (RVS). In this example, the RVS originally contains tuple "a". When a DVS containing tuples "b" and "c" is generated, it is then packed into the RVS, see Figure 3(a). Now RVS contains 3 tuples. The next generated DVS contains 2 tuples "d" and "e". Since the total number of tuples in RVS and DVS is larger than the vector width, DVS is then filled up with the last 2 tuples in RVS and forms a fully vectorized state.

2.5 Summary

Table 2 summarizes the characteristics of the four prefetching algorithms from several dimensions. First, in an iteration, the code stages executed by GP and SPP are both statically defined. In particular, GP executes a single code stage, while SPP executes all code stages in the code path. Second, for GP, AMAC and IMV, the buffer size of tuple states grows linearly with the number of tuples in a group. However, the buffer size for SPP is affected by the length of code paths. Then, both AMAC and IMV can efficiently handle divergent code paths. AMAC handles them by filling empty code stages, and IMV resolves them by reorganizing divergent vector states. Finally, IMV can take the advantage of SIMD instructions provided by modern processors.

3 IMPLEMENTATIONS

In this section, we present how we implement the prefetching algorithms on GPU. We first introduce the hardware architecture of GPU and APIs used for implementing software prefetching in Section 3.1. Then we illustrate our implementations for each algorithm on GPU in Section 3.2.

3.1 GPU Architecture and APIs for Prefetching

In order to implement prefetching on GPU, it is critical to understand the GPU architecture and the APIs provided for software prefetching. Figure 4(a) shows a simplified architecture of a GPU. Computations on GPU are performed by a number of Streaming Multiprocessors (SMs), each running multiple threads concurrently. GPU threads execute in Single Instruction Multiple Threads (SIMT)

```
T *data; // address
_mm_prefetch(data,1);
// computations...
Read(data);
```

```
1 T *data; // address
2 __shared__ T buffer;
3 P = cuda::make_pipeline();
4 cuda::memcpy_async
5   (&buffer,data,sizeof(T),P);
6 P.producer_commit();
7 // computations...
8 P.consumer_wait();
9 Read(&buffer);
```

Figure 5: Prefetching on CPU (left) vs. GPU (right).

model [3]. The memory of GPU is hierarchical. The largest and slowest memory is the global memory, which has a latency of about 400 cycles in our internal testing. All accesses to global memory are cached in L2 cache and L1 cache. The L2 cache is shared by all SMs with a latency of about 200 cycles. Compared to L2 cache and global memory, L1 cache is local to each SM and has a much lower latency of about 30 cycles. The shared memory physically resides with L1 cache on the same chip, but is fully controlled by users. During a standard memory read operation, data is fetched from global memory, then passes through the L2 and L1 caches, before finally being loaded into the registers for processing. Since the entire path is synchronized, it incurs a substantial latency.

In this paper, we leverage `cuda::memcpy_async` to implement software prefetching. This API is fully supported after NVIDIA Ampere architecture. It asynchronously copies data from the slow global memory to the fast shared memory, thereby avoiding costly memory stalls in the execution of the current thread, see Figure 4(a). These asynchronous memory requests can be managed by `cuda::pipeline` into an FIFO queue, as Figure 4(b) shows. A group of asynchronous memory requests are added to the queue by `producer_commit`. By calling `consumer_wait`, a thread explicitly waits for the oldest group in the queue to complete and removes it from the queue.

3.2 Implementation of Prefetching on GPU

We first introduce the general idea to implement prefetching on GPU. Figure 5 shows the different code between CPU-based and GPU-based prefetching. First, a buffer in shared memory is required to store the prefetched data (see the yellow part). Then, the prefetching is launched as an asynchronous memory copy from global memory to this buffer (see the green part). The memory copy is explicitly synchronized before subsequent accesses to this buffer (see the blue part).

Implementation of GP, SPP and AMAC. The GP, SPP and AMAC can be implemented on GPU by directly applying the general idea we previously discussed.

Implementation of IMV. Applying the general idea is not enough to implement IMV on GPU. The hard parts are (i) how to enable vectorization and (ii) how to efficiently reorganize vector states.

SIMD instructions are used to implement vectorization on CPU. Although GPU does not provide such instructions, its SIMT execution model matches with vectorization. Specifically, each 32 GPU threads are organized into a warp and execute together in parallel. Like SIMD, threads within a warp are restricted to issue the same instruction at a time on different data, thus any divergence can degrade the performance. Hence, we implement vectorization by distributing elements in a vector to all threads in a warp.

```
1  int prefix_mask = 0xffffffff >> (32 - lane);
2  __shared__ T RVS[32]; // RVS in shared memory
3  int rvs_n; // number of tuples in RVS
4
5  void ReorganizeVectorStates(T &DVS) {
6    int dvs_mask = __ballot_sync(0xffffffff, DVS);
7    int dvs_n = __popc(dvs_mask);
8    if (dvs_n + rvs_n >= 32) { // fill DVS
9      // offset of tuple in RVS to fetch
10     int offset = __popc(~dvs_mask & prefix_mask);
11     rvs_n += dvs_n - 32;
12     if (!DVS) DVS = RVS[rvs_n + offset];
13   } else { // pack DVS to RVS
14     // offset of empty slot in RVS to write
15     int offset = __popc(dvs_mask & prefix_mask);
16     if (DVS) RVS[rvs_n + offset] = DVS;
17     rvs_n += dvs_n;
18   }
19 }
```

Figure 6: Reorganization of vector states on GPU

However, it is still not trivial to efficiently reorganize the vector states in IMV. To address this challenge, we devise an implementation that leverages the efficient warp primitives and shared memory, which is inspired by the lane-refill operator in DogQC [17]. First, a mask of tuples in DVS is computed first by `__ballot_sync` (line 6), which sets the n-th significant bit to 1 if the n-th thread has tuples to process. Then the number of tuples in DVS is computed by `__popc` (line 7) which counts the valid bits in the mask. Next, if tuples in DVS and RVS can fill up a full vector (line 8), DVS is filled by fetching tuples from RVS in parallel (line 9-12), corresponding to the example in Figure 3(a). Otherwise tuples in DVS are packed into the empty slots in RVS in parallel (line 13-17), corresponding to the example in Figure 3(b).

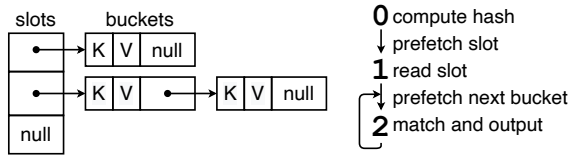
4 EVALUATED WORKLOAD

To study the capability of software prefetching on GPU, we evaluate it on two typical workloads in query processing, i.e., hash join probe and BTree search. In this section, we introduce our implementations and optimizations of them on GPU.

4.1 Hash Join Probe

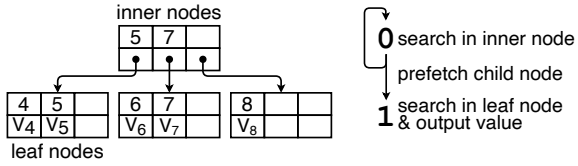
A lot of works study the efficient hash join algorithms on GPUs [33, 40, 42, 45]. In this work, we study the prefetching on the probe phase of the non-partitioned hash join, which is the most widely used join algorithm in GPU databases. We adapt Sioulas's [40] implementation and further improve it with early materialization to reduce random global memory accesses. As Figure 7(a) shows, the hash table has a slot array, where each slot stores the header of a bucket chain for all tuples that map to this slot. Each bucket contains the key-value of a tuple and the address of the next bucket.

State Machine. An execution of a state machine is a code path in Figure 2. The state machine of prefetching on hash join probe has 3 code stages, see Figure 7(b). The first stage loads a tuple and computes its hash value, then prefetches a slot according to the hash value. The second stage reads the slot and prefetches the first



(a) Hash table structure (b) State machine

Figure 7: Implementation of hash join probe



(a) BTree structure (b) State machine

Figure 8: Implementation of BTree search

bucket. The third stage compares the input key with the key in the bucket, outputs the result, and prefetches the next bucket. The third stage repeats until there is no next bucket, thus divergence of variable-length code paths may occur after the second and the third stages.

Optimization: State Cache. There is a lot of state information in hash join that is frequently accessed, e.g., the input tuples of ongoing probings. This locality can be used to reduce cache miss by caching the states. A straightforward implementation is declaring an array to store these states as a temporary variable. However, we observed from experiments that such an implicit approach can not guarantee the locality since (i) arrays on GPU are saved in the local memory (physically resides with global memory) instead of registers, and (ii) the L1 and L2 cache can not efficiently cache this array as it is easily polluted under the massive concurrent memory accesses on GPU. Given this observation, we further optimize our implementation by explicitly caching the states in shared memory.

4.2 BTree Search

GPU indexes are widely studied as it can provide a significant throughput for locating and accessing the data [5, 6, 14, 38, 48]. In this work, we study prefetching on BTree, one of the most popular indexes in databases. Specifically, we adapted Viktor's implementation [27, 43]. A simplified structure is shown in Figure 8(a). There are two types of nodes in a BTree, inner nodes and leaf nodes. Both of them contain a type identifier, a lock, and a key array. The difference is that an inner node contains an array of pointers to children, while a leaf node contains an array of values. We set the fan-out of nodes to 14 in default, which makes the size of a node equal to the size of an L1 cache line, i.e., 128 bytes, as previous work did [5, 6].

State Machine. Figure 8(b) shows the state machine of BTree search with 2 code stages. The first stage searches the input key in an inner node with a binary search. Given the search result, it prefetches the corresponding child node. The first stage repeats until it reaches a leaf node. It then enters into the second stage which searches the input key in the leaf node and outputs the result. Unlike hash join probe, searching a BTree will not result in variable-length code paths since all values reside in the same level.

Optimization 1: State Cache. Like the hash join probe in Section 4.1, there are also states that can be cached during the BTree search, e.g., the key to search. Therefore, we also optimize the BTree search by caching the states in shared memory.

Optimization 2: Reducing Divergence Inside Code Stages: Although there is no divergence in the length of different code paths, divergences still exist because the same stages in different code paths may cost different steps in the binary search. A way to fully reduce such divergence is enabling only 1 thread in each warp, but it significantly degrades the parallelism. Therefore, we trade off the divergence and parallelism by enabling 8 threads in each warp, which achieves the best performance in our experiments.

5 EXPERIMENTAL EVALUATION

In this section, we conduct experiments to evaluate the performance of software prefetching on GPU query processing. We first introduce the settings of our experiments in Section 5.1. Then, we analyze the performance on hash join probe and BTree search in Section 5.2 and Section 5.3. Finally we study how the number of concurrent threads affects the performance in Section 5.4. The source code has been made available at [4].

5.1 Experimental Setting

Datasets. We use a workload generator that has been used for several previous studies of join [7, 23, 40]. It generates a table that contains tuples with 4-byte keys and 4-byte values. The hash join in the experiments joins such two tables on their keys. The number of slots in hash table is set to the build table size. The BTree is built with tuples in such a table and searched with their keys.

Environments. Our experiments are conducted on a Ubuntu server with two Intel(R) Xeon(R) Gold 5318Y CPUs (2.10GHz) and an NVIDIA A10 GPU. The GPU has 72 SMs, each with 128 cores and 128 KB combined L1 cache. In such an L1 cache, a maximum of 100 KB can be dedicated to the shared memory. All SMs share 6 MB L2 cache and 24 GB device memory with a maximum bandwidth of 600 GB/s. All experiments are compiled and run with CUDA 12.1.

Compared Methods. We compare 5 methods for both hash join probe and BTree search. The baseline simply relies on hardware scheduling and does not use prefetching (denoted as BL). The other four are GP, SPP, AMAC and IMV. For each method, we tune the kernel launching parameters to achieve its best performance.

Algorithm Parameters. For hash join probe, the *group size G* of GP, AMAC and IMV is set to 4. The *prefetch distance D* of SPP is set to 1, and the pipeline depth is set to 3, which is the minimal length of its code paths. For BTree search, the *G* of GP, AMAC and IMV is set to 10. The *D* of SPP is set to 1 and the pipeline depth is set to the depth of the BTree, which is the length of code paths when searching the BTree. For some experiments, the shared memory capacity can not satisfy these parameters. In these cases, for GP, AMAC and IMV we lower the *G* as it only slightly degrades the performance. For SPP we skip these cases in the experiments.

5.2 Evaluation on Hash Join Probe

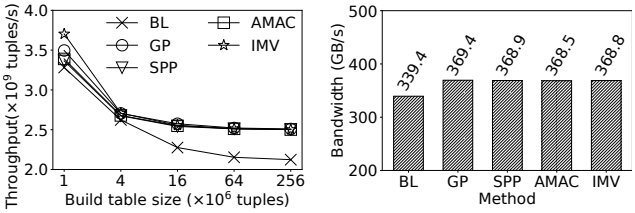
In this section, we measure the kernel time for the probe phase of hash join. Metrics like memory bandwidth and average active threads per warp are collected by Nsight Compute.

Performance on Uniform Workload. We first compare the throughput of hash join probe on a uniform workload. Keys in both the build table and probe table are unique and uniformly distributed. Therefore, there is no collision in the build phase and each probing only needs two random memory accesses, one to the slot and one to the bucket. We set the probe table size to 512M tuples and vary the size of the build table. As reported in Figure 9(a), all prefetch methods outperform the baseline when the build table contains more than 4M tuples. This performance gap increases as the build table grows and prefetch methods can gain up to 1.19X speedup. When the build table contains less than 4M tuples, all methods have similar performances. This is because a large part of hash table can fit in L2 cache, thus the memory access overhead is small for all methods.

To further understand the reason for this performance gain, we measure the achieved memory bandwidth for hash join probe. We observed that the prefetch methods can achieve a bandwidth of about 1.09X that of BL, see Figure 9(b). The increased memory bandwidth is the key to the success of prefetch methods since hash join probe is memory-bandwidth bound.

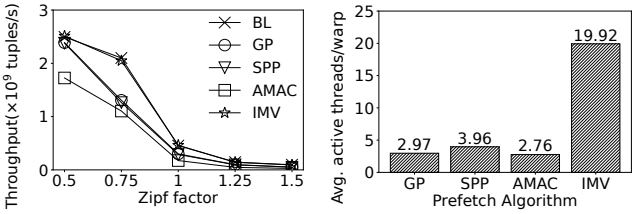
Performance on Skewed Workload. We next investigate the performance under the effect of skewness. Keys in the build table is generated with Zipf distribution. keys in the probe table remain uniform. Both tables contain 512 M tuples. In this case, the lengths of bucket chains are diverse and result in divergence during probing. As reported in Figure 10(a), the BL performs similarly to IMV and outperforms other prefetch methods. For an extremely skewed join (skew factor > 1), all methods perform badly due to the low parallelism when probing the long tail. This experiment shows that (i) the hardware scheduling is good enough at handling divergence and (ii) IMV successfully reduces the divergence of prefetching under GPU's SIMT execution like it did under CPU's SIMD execution. Another interesting finding is that AMAC, which was also proposed to eliminate such divergence, performs even worse than GP and SPP. Its optimization that fills an empty stage with a new stage does not address but worsen the problem of divergence of SIMT. Such a new stage is different from the ongoing stages of other threads within the same warp, therefore they can not execute together in parallel. Even worse, all its later stages may diverge. To further verify our analysis, we measure the average number of active threads per warp per GPU cycle. Fewer active threads indicate a higher divergence in SIMT. As Figure 10(b) shows, IMV enjoys the lowest divergence while AMAC suffers the most significant divergence.

Effect of Optimization. Figure 11 shows the effect of state cache in shared memory by comparing the throughput between caching the state array in a temporary variable (denoted as TEMP) and in shared memory (denoted as SMEM) during the hash join probe. SPP, AMAC and IMV all benefit from this optimization and gain 1.28X, 1.18X and 1.37X speedup. However, GP has no performance gain because its state array is compiled into registers. Compared to other methods, GP accesses the states with smaller iterations



(a) Throughput (b) Achieved memory bandwidth

Figure 9: Hash join probe on uniform workload



(a) Throughput (b) Divergence in SIMT

Figure 10: Hash join probe on skewed workload

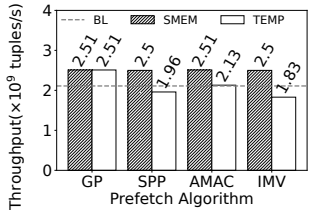
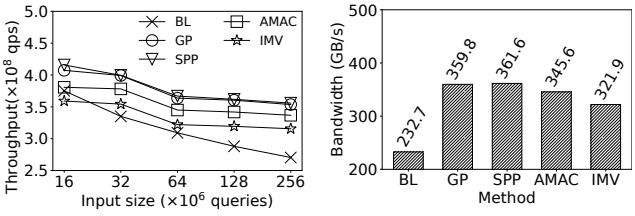


Figure 11: Effect of state cache on hash join probe



(a) Throughput (b) Achieved memory bandwidth

Figure 12: Performance evaluation of BTree search

containing only one static code stage. These simple iterations are easily unrolled by the compiler, and their indexed accesses to the state array are also unrolled into separated accesses to each register.

5.3 Evaluation on BTree Search

In this section, we measure the kernel time for searching the BTree and use Nsight Compute to collect performance metrics.

Performance Comparison. The throughputs of searching a BTree with different methods are shown in Figure 12(a). Obviously all prefetch methods outperform the BL. Among them, GP and SPP have the best performance and both of them achieve 1.31X speedup

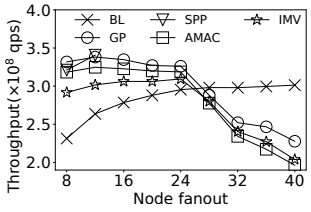


Figure 13: Impact of node fanout on BTree search

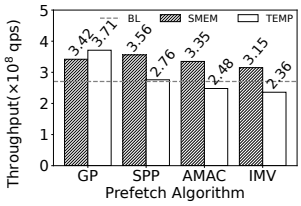


Figure 14: Effect of state cache on BTree search

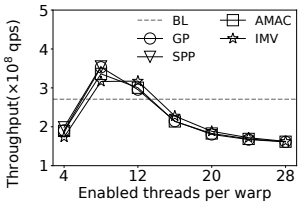


Figure 15: Trade-off between divergence and parallelism

over BL. AMAC achieves a smaller speedup of 1.24X due to the extra costs of managing the states. IMV gains the minimal speedup of 1.16X due to the expensive warp synchronization and divergence checking in every stage. As Figure 12(b) shows, the comparison of achieved memory bandwidths also matches the performance.

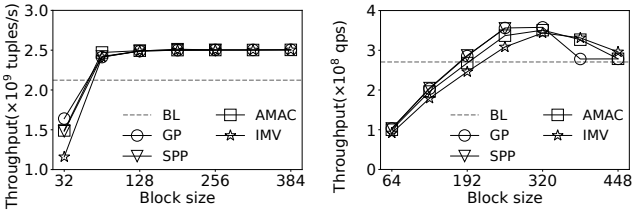
Impact of Node Fanout. In practice, people choose the node fanout according to their workload characteristics. A larger fanout usually indicates more conflicts in concurrent insertion but shorter search paths. In this experiment, we study the search performance of searching 256M keys with different fanouts. As Figure 13 shows, the maximum throughput of prefetch methods is higher than its of BL. When fanout is not greater than 24, all prefetch methods outperform BL. However, when the fanout is larger than 24, throughputs of prefetch methods dramatically drop as the fanout grows. The reason is that prefetching the entire nodes results in significant read amplification.

Effect of Optimizations. We first study the effect of the state cache in shared memory. SPP, AMAC and IMV can gain 1.28X, 1.18X and 1.37X speedup with this strategy, see figure 14. However, it degrades 7% of GP's throughput because the state array is compiled into the faster registers in default, as we analyzed in Section 5.2.

Then we study how the number of enabled threads inside a warp impacts the performance. The best performance is achieved by enabling 8 threads per warp, see Figure 15. Enabling too many threads can exacerbate the divergence, while enabling too few threads can lower the parallelism, and both degrade the performance.

5.4 Impact of Concurrent Threads

The performance of a CUDA kernel is influenced by the number of concurrent threads. When there are more concurrent threads than GPU cores, the hardware scheduler switches among these threads to overlap memory accesses and compute operations. We conduct an experiment to study its impact on GPU prefetching. To ensure that all threads are running concurrently, we fix the



(a) Hash join probe (b) BTree search

Figure 16: Impact of concurrent threads (72 blocks)

number of blocks to the number of SMs and vary the number of threads in a block, i.e., block size. As Figure 16(a) shows, the hash join probe needs at least 64 concurrent threads in an SM to get the best performance. Interestingly, it only uses half of the available GPU cores. In this case, hardware scheduling is not enabled. The performance is achieved only with software prefetching. However, BTree search achieves the best throughput with 256 threads, which is larger than the number of cores. In this case, simply relying on neither software prefetching nor hardware scheduling can get the best performance. Only a proper combination of them can efficiently utilize the memory bandwidth.

6 RELATED WORK

In this section, we discuss relevant studies on query processing with software prefetching and prefetching on GPU.

6.1 Software Prefetching on Query Processing

Software prefetching has long been studied and is now a standard optimization to deal with cache miss in CPU databases. In this paper, we choose to study GP, SPP, AMAC and IMV because (i) these algorithms are generic, thus can be adapted to all kinds of workloads, and (ii) they do not rely on specific runtime or library such as coroutine, thus can be implemented on GPU. Besides them, a lot of studies have been proposed to enhance software prefetching for specific tasks in query processing [11, 19, 20, 32, 49]. For example, pB$^+$-Tree [11] uses prefetching to effectively create wider nodes and reduce the tree height of B$^+$-Tree. CoroBase [19] is an OLTP engine with the coroutine-to-transaction model, which benefits from software prefetching and avoids application changes. CoroGraph [49] integrates efficient software prefetching and scheduling to enhance both cache and work efficiency in graph processing.

6.2 GPU Prefetching

Many studies leverage prefetching to improve the performance of memory-bound GPU applications [21, 25, 26, 30, 35, 37]. Most of them focus on the hardware and compiler levels. MT-prefetching [26] predicts patterns of future memory access with intra-warp and inter-warp strides. CAPS [25] contains a CTA-aware prefetcher and scheduler that consider the trade-off between accuracy and prefetch distance. Snake [30] issues prefetching requests based on chains of strides among the consecutive load instructions at different levels. However, these studies have limitations in incorporating user knowledge to better optimize for specific workloads. Fortunately, the NVIDIA Ampere architecture brings fully controllable APIs for software prefetching, which opens the door for users to

design and implement their own prefetching policies for various workloads [1, 2]. Note that we are the first to explore the ability of GPU prefetching in the database area.

7 LESSONS LEARNT AND CONCLUSION

In this paper, we study software prefetching on GPU query processing. Specifically, we implement hash join probe and BTree search on GPU with four prefetch algorithms, i.e., GP, SPP, AMAC, and IMV. Experiments demonstrate that prefetching can accelerate GPU query processing by achieving a higher memory bandwidth. However, such a performance is not trivial to gain. We summarize a brief guideline in Table 3 about how to efficiently use prefetching in GPU query processing, according to our empirical experiments on NVIDIA A10 GPU. Note that these optimizations and design choices also apply to other GPUs although the optimal parameters may differ. In addition, we conclude several insights to take away.

- **For workloads with divergent code paths, e.g., hash join probe, IMV is the most efficient.** With the efficient GPU implementation we proposed, IMV can minimize the divergence within the warp. By contrast, both GP and SPP lack the ability to reduce divergence. AMAC was originally proposed to alleviate the divergence of prefetching on CPU. However, it instead exacerbates the divergence under the SIMT execution of GPU.
- **For workloads with uniform code paths, e.g., BTree search, GP is the most efficient.** In contrast, both IMV and AMAC incur extra compute overhead to check the divergence. SPP has similar performance to GP. However, it consumes more memory to buffer the prefetching states. In SPP, the size of state buffer grows linearly with the length of code paths.
- **For workloads with divergence within a code stage, e.g., in-node binary search in BTree, the best performance requires enabling a proper number of threads per warp.** Enabling more threads per warp brings higher parallelism but can also aggravate the thread divergence within the warp. There is a balance to strike between parallelism and divergence.
- **The performance can be improved by explicitly caching the prefetching states in registers and shared memory.** Under the default behaviour of the compiler, arrays used to buffer the prefetching states are likely to be stored in the slow global memory, which is inefficient to access. The problem can be solved by explicitly moving them to registers and shared memory.
- **Software prefetching and hardware scheduling can work together.** Software prefetching and hardware scheduling are two orthogonal methods to improve the MLP. For specific workloads such as BTree search, a proper combination of both methods performs better than enabling either one of them alone.

This paper points in a new direction to optimize query processing on GPUs. Given the valuable insights about the ability of GPU prefetching, it is promising to accelerate other analytic tasks in data management in this way. In addition, there are challenging topics that worth further research, e.g., utilizing GPU prefetching for workloads with memory writes, and developing efficient query processing models that leverages the power of prefetching for GPUDBs.

Table 3: Guideline for prefetching on GPU query processing

Implementation	Hash Join Probe	BTree Search
Algorithm	IMV	GP
Enabled threads/warp	32	8
State array	shared memory	temporary variable
Block size	$\geq \frac{1}{2}\times$ cores per SM	$2\times$ cores per SM

ACKNOWLEDGMENTS

We thank all reviewers for their constructive feedback to help us improve the quality of this paper. This work is partially supported by Shenzhen Fundamental Research Program (Grant No. 20220815112848002), the Guangdong Provincial Key Laboratory (Grant No. 2020B121201001) and a research gift from Huawei Gauss department. Dr. Bo Tang is also affiliated with the Research Institute of Trustworthy Autonomous Systems, Southern University of Science and Technology, Shenzhen, China.

REFERENCES

[1] 2020. *Controlling Data Movement to Boost Performance on the NVIDIA Ampere Architecture.* https://developer.nvidia.com/blog/controlling-data-movement-to-boost-performance-on-ampere-architecture
[2] 2022. *Boosting Application Performance with GPU Memory Prefetching.* https://developer.nvidia.com/blog/boosting-application-performance-with-gpu-memory-prefetching
[3] 2024. *Single instruction, multiple threads.* https://en.wikipedia.org/wiki/Single_instruction,_multiple_threads
[4] 2024. *Source code.* https://github.com/DBGroup-SUSTech/GPUDB-Prefetch
[5] Muhammad A. Awad, Saman Ashkiani, Rob Johnson, Martín Farach-Colton, and John D. Owens. 2019. Engineering a high-performance GPU B-Tree. In *PPoPP*. 145–157.
[6] Muhammad A. Awad, Serban D. Porumbescu, and John D. Owens. 2023. A GPU Multiversion B-Tree. In *PACT*. 481–493.
[7] Cagri Balkesen, Gustavo Alonso, Jens Teubner, and M. Tamer Özsu. 2013. Multi-core, main-memory joins: sort vs. hash revisited. *Proc. VLDB Endow.* 7, 1 (2013), 85–96.
[8] Nils Boeschen and Carsten Binnig. 2022. GaccO - A GPU-accelerated OLTP DBMS. In *SIGMOD*. 1003–1016.
[9] Jiashen Cao, Rathijit Sen, Matteo Interlandi, Joy Arulraj, and Hyesoon Kim. 2023. GPU Database Systems Characterization and Optimization. *Proc. VLDB Endow.* 17, 3 (2023), 441–454.
[10] S. Chen, A. Ailamaki, P.B. Gibbons, and T.C. Mowry. 2004. Improving hash join performance through prefetching. In *ICDE*. 116–127.
[11] Shimin Chen, Phillip B. Gibbons, and Todd C. Mowry. 2001. Improving index performance through prefetching. *SIGMOD Rec.* 30, 2 (2001), 235–246.
[12] Periklis Chrysogelos, Manos Karpathiotakis, Raja Appuswamy, and Anastasia Ailamaki. 2019. HetExchange: encapsulating heterogeneous CPU-GPU parallelism in JIT compiled engines. *Proc. VLDB Endow.* 12, 5 (2019), 544–556.
[13] Periklis Chrysogelos, Panagiotis Sioulas, and Anastasia Ailamaki. 2019. Hardware-conscious Query Processing in GPU-accelerated Analytical Engines. In *Conference on Innovative Data Systems Research*.
[14] Yangshen Deng, Muxi Yan, and Bo Tang. 2024. Accelerating Merkle Patricia Trie with GPU. *Proc. VLDB Endow.* 17, 8 (2024), 1856–1869.
[15] Zhuhe Fang, Beilei Zheng, and Chuliang Weng. 2019. Interleaved multi-vectorizing. *Proc. VLDB Endow.* 13, 3 (2019), 226–238.
[16] Henning Funke, Sebastian Breß, Stefan Noll, Volker Markl, and Jens Teubner. 2018. Pipelined Query Processing in Coprocessor Environments. In *SIGMOD*. 1603–1618.
[17] Henning Funke and Jens Teubner. 2020. Data-parallel query processing on non-uniform data. *Proc. VLDB Endow.* 13, 6 (2020), 884–897.
[18] Bingsheng He and Jeffrey Xu Yu. 2011. High-throughput transaction executions on graphics processors. *Proc. VLDB Endow.* 4, 5 (2011), 314–325.
[19] Yongjun He, Jiacheng Lu, and Tianzheng Wang. 2020. CoroBase: coroutine-oriented main-memory database engine. *Proc. VLDB Endow.* 14, 3 (nov 2020), 431–444.
[20] Kaisong Huang, Tianzheng Wang, Qingqing Zhou, and Qingzhong Meng. 2023. The Art of Latency Hiding in Modern Database Engines. *Proc. VLDB Endow.* 17, 3 (2023), 577–590.

[21] Adwait Jog, Onur Kayiran, Asit K. Mishra, Mahmut T. Kandemir, Onur Mutlu, Ravishankar Iyer, and Chita R. Das. 2013. Orchestrated scheduling and prefetching for GPGPUs. In *ISCA*. 332–343.

[22] Tomas Karnagel, René Müller, and Guy M Lohman. 2015. Optimizing GPU-accelerated Group-By and Aggregation. *ADMS@ VLDB* 8 (2015), 20.

[23] Changkyu Kim, Tim Kaldewey, Victor W. Lee, Eric Sedlar, Anthony D. Nguyen, Nadathur Satish, Jatin Chhugani, Andrea Di Blas, and Pradeep Dubey. 2009. Sort vs. Hash revisited: fast join implementation on modern multi-core CPUs. *Proc. VLDB Endow.* 2, 2 (2009), 1378–1389.

[24] Onur Kocberber, Babak Falsafi, and Boris Grot. 2015. Asynchronous memory access chaining. *Proc. VLDB Endow.* 9, 4 (2015), 252–263.

[25] Gunjae Koo, Hyeran Jeon, Zhenhong Liu, Nam Sung Kim, and Murali Annavaram. 2018. CTA-Aware Prefetching and Scheduling for GPU. In *IPDPS*. 137–148. https://doi.org/10.1109/IPDPS.2018.00024

[26] Jaekyu Lee, Nagesh B. Lakshminarayana, Hyesoon Kim, and Richard Vuduc. 2010. Many-Thread Aware Prefetching Mechanisms for GPGPU Applications. In *MICRO*. 213–224.

[27] Viktor Leis, Michael Haubenschild, and Thomas Neumann. 2019. Optimistic Lock Coupling: A Scalable and Efficient General-Purpose Synchronization Method. *IEEE Data Eng. Bull.* 42 (2019), 73–84.

[28] Haotian Liu, Bo Tang, Jiashu Zhang, Yangshen Deng, Xiao Yan, Xinying Zheng, Qiaomu Shen, Dan Zeng, Zunyao Mao, Chaozu Zhang, Zhengxin You, Zhihao Wang, Runzhe Jiang, Fang Wang, Man Lung Yiu, Huan Li, Mingji Han, Qian Li, and Zhenghai Luo. 2022. GHive: accelerating analytical query processing in apache hive via CPU-GPU heterogeneous computing. In *SoCC*. 158–172.

[29] Tobias Maltenberger, Ivan Ilic, Ilin Tolovski, and Tilmann Rabl. 2020. Evaluating Multi-GPU Sorting with Modern Interconnects. In *SIGMOD*. 1795–1809.

[30] Saba Mostofi, Hajar Falahati, Negin Mahani, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2023. Snake: A Variable-length Chain-based Prefetching for GPUs. In *MICRO*. 728–741.

[31] Todd Carl Mowry. 1995. *Tolerating latency through software-controlled data prefetching*. Ph. D. Dissertation.

[32] Jan Mühlig and Jens Teubner. 2021. MxTasks: How to Make Efficient Synchronization and Prefetching Easy. In *SIGMOD*. 1331–1344.

[33] Johns Paul, Bingsheng He, Shengliang Lu, and Chiew Tong Lau. 2019. Revisiting Hash Join on Graphics Processors: A Decade Later. In *ICDEW*. 294–299.

[34] Johns Paul, Bingsheng He, Shengliang Lu, and Chiew Tong Lau. 2020. Improving execution efficiency of just-in-time compilation based query processing on GPUs. *Proc. VLDB Endow.* 14, 2 (2020), 202–214.

[35] Mohammad Sadrosadati, Amirhossein Mirhosseini, Seyed Borna Ehsani, Hamid Sarbazi-Azad, Mario Drumond, Babak Falsafi, Rachata Ausavarungnirun, and Onur Mutlu. 2018. LTRF: Enabling High-Capacity Register Files for GPUs via Hardware/Software Cooperative Register Prefetching. In *ASPLOS*. 489–502.

[36] Nadathur Satish, Mark Harris, and Michael Garland. 2009. Designing efficient sorting algorithms for manycore GPUs. In *2009 IEEE International Symposium on Parallel & Distributed Processing*. 1–10.

[37] Ankit Sethia, Ganesh Dasika, Mehrzad Samadi, and Scott Mahlke. 2013. APOGEE: adaptive prefetching on GPUs for energy efficiency. In *PACT*. 73–82.

[38] Amirhesam Shahvarani and Hans-Arno Jacobsen. 2016. A Hybrid B+-tree as Solution for In-Memory Indexing on CPU-GPU Heterogeneous Computing Platforms. In *SIGMOD*. 1523–1538.

[39] Anil Shanbhag, Samuel Madden, and Xiangyao Yu. 2020. A Study of the Fundamental Performance Characteristics of GPUs and CPUs for Database Analytics. In *SIGMOD*. 1617–1632.

[40] Panagiotis Sioulas, Periklis Chrysogelos, Manos Karpathiotakis, Raja Appuswamy, and Anastasia Ailamaki. 2019. Hardware-Conscious Hash-Joins on GPUs. In *ICDE*. 698–709.

[41] Elias Stehle and Hans-Arno Jacobsen. 2017. A Memory Bandwidth-Efficient Hybrid Radix Sort on GPUs. In *SIGMOD*. 417–432.

[42] Lasse Thostrup, Gloria Doci, Nils Boeschen, Manisha Luthra, and Carsten Binnig. 2023. Distributed GPU Joins on Fast RDMA-capable Networks. *Proc. ACM Manag. Data* 1, 1, Article 29 (2023), 26 pages.

[43] Ziqi Wang, Andrew Pavlo, Hyeontaek Lim, Viktor Leis, Huanchen Zhang, Michael Kaminsky, and David G. Andersen. 2018. Building a Bw-Tree Takes More Than Just Buzz Words. In *SIGMOD*. 473–488.

[44] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM* 52, 4 (2009), 65–76.

[45] Bowen Wu, Dimitrios Koutsoukos, and Gustavo Alonso. 2023. Efficiently Processing Large Relational Joins on GPUs. arXiv:2312.00720 [cs.DB]

[46] Bobbi Yogatama, Brandon Miller, Yunsong Wang, Graham Markall, Jacob Hemstad, Gregory Kimball, and Xiangyao Yu. 2023. Accelerating User-Defined Aggregate Functions (UDAF) with Block-wide Execution and JIT Compilation on GPUs. In *DaMoN*. 19–26.

[47] Bobbi W. Yogatama, Weiwei Gong, and Xiangyao Yu. 2022. Orchestrating data placement and query execution in heterogeneous CPU-GPU DBMS. *Proc. VLDB Endow.* 15, 11 (2022), 2491–2503.

[48] Weihua Zhang, Chuanlei Zhao, Lu Peng, Yuzhe Lin, Fengzhe Zhang, and Yunping Lu. 2023. Boosting Performance and QoS for Concurrent GPU B+trees by Combining-Based Synchronization. In *PPoPP*. 1–13.

[49] Xiangyu Zhi, Xiao Yan, Bo Tang, Ziyao Yin, Yanchao Zhu, and Minqi Zhou. 2024. CoroGraph: Bridging Cache Efficiency and Work Efficiency for Graph Algorithm Execution. *Proc. VLDB Endow.* 17, 4 (2024), 891–903.